

AgentCom: A Bounded Two-Agent Dialogue System

KIUA2003 / KIUA2005-1, Final Project Report

1. Design

Scenario

Scenario type: Debate. Topic: "Solo Leveling is the greatest manhwa of all time." I picked a debate because it maps cleanly onto two clearly opposed roles, and because I could write personas I actually cared about, which made it easier to judge whether the model's output was any good.

Personas

SungJinFan (argues FOR):

"Debate topic: Solo Leveling is the greatest manhwa of all time. You are an obsessive Solo Leveling fan. You argue IN FAVOUR of the topic. Back up your points with specifics: the art and action choreography, Sung Jin-Woo's power progression from E-rank to Shadow Monarch, and how his arc reflects real struggle and growth. Make one concrete point per turn. Be brief: at most 2 sentences. Directly address the other speaker's last point rather than repeating yourself."

TowerOfGodFan (argues AGAINST):

"Debate topic: Solo Leveling is the greatest manhwa of all time. You are a devoted Tower of God reader. You argue AGAINST the topic. Back up your points with specifics: Tower of God's layered worldbuilding, its mystery-driven plotting, and character depth that goes beyond raw power progression. Push back directly on claims that Solo Leveling's simplicity is a strength. Make one concrete point per turn. Be brief: at most 2 sentences. Directly address the other speaker's last point rather than repeating yourself."

"Goal reached": the measurable definition

Goal reached = a separate judge agent reads the full transcript after N rounds (8) and returns a score (1-5), a success flag, a one-sentence reason, and a winner. The judge's rubric checks two things a program can actually evaluate: (1) did each side use concrete details (character names, plot points) rather than vague claims, and (2) did each side directly rebut the other's previous point rather than repeating their own position or simply agreeing. This is detected by a single structured judge() call after the run completes, not during the run itself. See Section 3.

Engine mechanics (Week 2)

Each agent's view of the conversation is built by view_for(agent, transcript): the agent's own past lines are labelled "assistant" and the other agent's lines are labelled "user", with the agent's system prompt always first. This is the core subtlety the course brief calls out: the message format has no field for "who is speaking", only roles, so each agent needs its own relabelled copy of the same neutral transcript. DialogueEngine.run() alternates speakers via next_speaker() (turn index modulo agent count), builds each speaker's view, calls the model, records the reply as an Entry, and updates the Budget every turn.

Safety: three independent budgets

Every call to the model happens inside a while loop guarded by a single Budget object enforcing max_turns, max_tokens, and max_seconds simultaneously, per the course's hard rule that no such loop may exist without a Budget controlling it. Any one limit alone is not sufficient: a turn cap does not protect against a single turn generating thousands of tokens, and a token cap does not protect against a model that hangs. I verified this is not just a design intention but an actual working property of the code with test_budgets.py, which deliberately tightens one limit at a time (leaving the other two effectively unreachable) and confirms budget.stop_reason correctly names whichever limit fired: max_turns, max_tokens, and max_seconds each independently stopped a run in isolation.

Example: an excerpt from a real run

From transcripts/manhwa_debate.json (temperature 0.7), turns 0-1, showing the role-mapping and personas working as intended against a real model:

SungJinFan: "I firmly believe Solo Leveling is the greatest manhwa of all time, and its art and action choreography are a significant reason why..."

TowerOfGodFan: "While the art and action in Solo Leveling are undoubtedly impressive, this can be seen as a shallow attempt to compensate for a lack of depth in the story itself..."

This shows the two agents correctly holding opposing, persona-consistent positions from the very first exchange. This is direct evidence that view_for's role assignment and each agent's distinct system prompt are both working correctly against a real model, not just in the mock-mode unit tests.

2. Results Table

All values below are pulled directly from saved transcript files in transcripts/ using build_results_table.py, not from memory. The first three rows vary temperature only (0.3 / 0.7 / 1.2), everything else held fixed. The fourth row is the original single manhwa debate run. The fifth row is the deliberate failure-mode run (Section 3).

File	Temp	Turns	Tokens	Seconds	Stop reason	Score	Success	Winner
experiment_temp_0.3.json	0.3	8	4456	32.3	max_turns	4	True	tie
experiment_temp_0.7.json	0.7	8	3820	28.9	max_turns	4	True	tie
experiment_temp_1.2.json	1.2	8	4260	31.2	max_turns	4	True	tie
manhwa_debate.json	0.7	8	3465	36.3	max_turns	4	True	tie
failure_sycophancy.json	0.7 (identical prompts)	8	3859	34.7	max_turns	4	True	tie

What the experiment shows

Varying temperature from 0.3 to 1.2 produced NO measurable difference in judge score, success, or winner. All three runs scored 4/True/tie. This is a null result, reported honestly rather than invented: the course brief explicitly states a null result honestly reported beats a difference invented from noisy runs.

However, reading the actual transcripts (not just the judge's number) shows temperature DID affect style, which the coarse 1-5 rubric didn't capture: at 0.3 the debate stayed tightly on-topic and purely adversarial throughout; at 0.7 it began drifting slightly (one reply compared Solo Leveling to unrelated series like Black God and Noblesse); at 1.2 two coherence issues appeared: a stray out-of-character line ("The stage is set, let's debate!") in the very first turn, and noticeably more hedging language ("That's a fair point, but...") in later turns, a step toward the sycophancy failure mode described in Section 3. This suggests the judge's numeric rubric is too coarse to detect stylistic drift even when it is visible on close reading, a limitation of the evaluation and not just of the dialogue.

A separate, temperature-independent observation: at every temperature, the model hallucinated Tower of God character names that do not exist in the real series (e.g. "Han Jisoo", "Chakho", "Park Yeon-jin", "Hansong and Shai"). Since this happened at all three temperatures, it is a base-model knowledge gap rather than a temperature effect. It is kept here as supporting evidence for the failure-mode analysis below.

3. Failure-Mode Analysis

To generate a genuine failure rather than a hypothetical one, I followed the course brief's own guidance that identical personas reliably produce sycophancy. I ran the same debate topic with both agents given the SAME neutral system

prompt ("discuss this topic thoughtfully"), removing the FOR/AGAINST framing that normally forces disagreement. Full transcript: transcripts/failure_sycophancy.json.

What happened

Turns 0-1 opened with real disagreement (AgentA argued philosophical themes were most important; AgentB argued world-building and the protagonist mattered more). By turns 2-3 the language started softening ("I agree that... However, I would argue..."). By turns 4-7 the debate had fully collapsed into agreement, as shown directly in the transcript:

Turn 4, AgentA: "I think you make a valid point that the themes in Solo Leveling are more explicitly meant to make a commentary on societal issues..."

Turn 5, AgentB: "I think that's a great way to articulate the author's intentions, while also acknowledging the potential impact on the reader's experience..."

Turn 7, AgentB: "...It's like the author is saying, 'I'm not just telling a story, I'm trying to make you think about the world we live in...'"

By the final turn AgentB is simply elaborating on AgentA's own framing rather than offering any opposing claim. No disagreement is left in the conversation at all. This is sycophancy: with no assigned opposing role, the two agents drifted toward validating each other instead of maintaining a debate.

The second, more interesting failure: the judge missed it

I ran this failed transcript through judge() expecting a low score or success=false. Instead it returned score=4, success=true, winner="tie", with the reason claiming "each side directly rebutting the other's previous point", which is false for the second half of the transcript. The rubric checks each turn locally for concrete details and rebuttal-shaped phrasing (words like "however", "I'd argue"), but never tracks whether the conversation's overall trajectory is converging toward agreement across turns. A debate that starts adversarial and ends in full agreement still passes, because each individual turn still contains a technically-rebutting phrase, even when the content of that phrase concedes the point.

This matches a known bias in LLM-as-judge setups from the course's own reading list (Zheng et al., "Judging LLM-as-a-Judge"): judges evaluating local features can miss global patterns like drift. Diagnosis: the rubric needs a trajectory-level check (e.g. comparing the stance of the first and last few turns, or an explicit sycophancy flag) rather than only turn-level phrasing checks, to catch this class of failure.

4. Cost and Time Accounting

Across five real 8-turn runs against llama3.2:3b via Ollama (temperatures 0.3, 0.7, 1.2, plus the original debate, plus the sycophancy run), token usage ranged from 3820 to 4456 tokens per run (mean approx. 3970), and wall-clock time ranged from 28.9 to 36.3 seconds (mean approx. 32.7s). This is roughly 480-560 tokens and 3.6-4.5 seconds per turn on average, on a local 3B-class model with no per-token cost. All runs stopped on max_turns (the turn cap), meaning the token and wall-clock budgets (4000 tokens, 120 seconds) were never the binding constraint for this scenario at 8 turns. Turn count was the limiting factor throughout. This was separately confirmed with test_budgets.py, which showed each of the three budgets (max_turns, max_tokens, max_seconds) CAN independently stop a run when deliberately made the tightest limit, even though in the real experiment runs it was consistently max_turns that fired first.

Because there is no per-token API cost with a local model, the real cost here is wall-clock time and local compute: roughly half a minute per 8-turn debate on this machine. Scaling to a longer conversation (e.g. 30+ turns, as tested in test_context.py) would increase this roughly linearly without context management, which is exactly why manage_context (truncation) was implemented. See Section 1 and week3_notes.md for the full justification.

5. Reflection: What a Framework Would Have Done

A framework like AutoGen or LangGraph would have handled turn-taking, message formatting, and role assignment out of the box, meaning I wouldn't have had to write `view_for()` myself or think carefully about why agent A's messages need to appear as "assistant" to itself and "user" to agent B. That would have saved time, but it also would have hidden the exact thing this course was about: understanding that a chat model has no built-in sense of who is speaking, and that role-mapping is something your code has to construct explicitly, every turn.

A framework would also likely have shipped a more sophisticated built-in evaluator or judge abstraction than the single `judge()` call I wrote by hand. But writing my own judge, and then watching it fail to catch the sycophancy collapse in Section 3, taught me something a framework's judge would have hidden: that a judge is only as good as its rubric, and that a rubric checking local turn features can systematically miss a global failure pattern. I would not have found or understood that limitation if the judge had been an opaque built-in component instead of code I wrote and could inspect.

Overall: a framework optimizes for getting a working multi-agent system quickly; building it by hand optimized for understanding exactly where the system is fragile, which was the actual goal of this course.

I did not attempt the stretch goals (tool-use agents, a third moderator agent, or mixed-model debates). Given the time available, I prioritized getting the required milestones (context management, a working judge, a real experiment, and a genuine diagnosed failure) solid and evidenced over adding scope on top. The most natural stretch goal for this scenario would have been a third judge-moderator agent that decides when to end the debate early rather than always running to `max_turns`, since the current judge only ever runs after the fact.

Limitations

A few honest limitations of this project, worth stating plainly rather than glossing over. First, all runs used a single 3B-class model (llama3.2:3b) for both agents and the judge; behavior may differ meaningfully with a larger model or with different models per role, which was not tested here. Second, the temperature experiment used one run per setting rather than multiple repeats, so the null result in Section 2 reflects three single samples, not an average across several runs at each temperature; a noisy individual run could look like "no difference" even if a real effect exists. Third, the judge's rubric is coarse (a single 1-5 integer plus a short reason) and, as Section 3 shows directly, can miss trajectory-level failures like a debate drifting into agreement over several turns even when every individual turn still looks locally reasonable. Fourth, this project covers only the Debate scenario; the engine itself is scenario-agnostic, but the judge rubric and goal-reached criterion are written specifically for adversarial debate and would need to be rewritten for a different scenario type such as Socratic tutor or negotiation.

Reproducing These Results

Setup: install Ollama, then pull the model with `ollama pull llama3.2:3b` before running anything without `--mock`. One command runs the main scenario: `python run.py --config configs/debate.yaml` (add `--mock` to run offline with canned replies for development, useful for checking the loop logic without waiting on real model calls). The experiment in Section 2 is reproduced with `python experiment.py --config configs/debate.yaml`, which runs all three temperatures and saves each as its own transcript. The failure run in Section 3 is reproduced with `python run.py --config configs/failure_sycophancy.yaml`. Every transcript is judged with `python run_judge.py transcripts/<file>.json`, which also writes the verdict back into that transcript file. The full results table is rebuilt from whatever is currently saved in `transcripts/` with `python build_results_table.py`. So every number in this report traces back to a file that can be regenerated and re-checked, not to memory.